

Otomotiv Mühendisliğinden Yazılıma Aktarılabilecek Dersler

ÖFY

July 2026

Giriş

Otomotiv mühendisliği, hatanın maliyetinin geç fark edildikçe katlanarak arttığı bir alandır – bir kalıp basıldıktan, bir seri üretim başladıktan sonra hatayı düzeltmek milyonlarca dolara mal olur. Bu yüzden endüstri, tasarım aşamasına orantısız yatırım yapmayı sistematik bir disiplin haline getirmiştir. Bu doküman, bu disiplinlerin yazılım mühendisliğine nasıl aktarılabileceğini altı ana eksenle toplar.

Güvenilirlik Mühendisliği

Tolerans Yığılması (Tolerance Stacking)

Mekanik mühendislikte her parçanın kendi toleransı vardır; parçalar birleştğinde bu toleranslar birikir ve sistem düzeyinde beklenmedik boşluklar veya sıkışmalar ortaya çıkabilir.

Worst-Case ve RSS Yöntemleri

Worst-case yöntemi: Tüm toleranslar aynı yönde en kötü uçlarına gitmiş varsayılır. Beş parça, her biri $\pm 0.1\text{mm}$ ise, toplam olası sapma $\pm 0.5\text{mm}$ 'dir. Çok muhafazakârdır ama garanti eder.

RSS (Root Sum Square) yöntemi: Bağımsız toleransların karelerinin toplamının karekökü alınır. Beş parça için: $\sqrt{0.1^2 \times 5} \approx \pm 0.22\text{mm}$. Daha gerçekçidir ama teorik olarak garantili değildir.

Yazılım Karşılığı

Her mikroservis kendi başına “makul” bir varsayımla tasarlanır (timeout süresi, retry sayısı, cache TTL, queue işleme süresi, rate limit). Tek tek test edildiğinde her biri doğrudur, ama bir istek zinciri boyunca art arda çalıştıklarında zincirleme etki hiçbir mühendisin tek başına öngörmediği bir davranış üretir – timeout dolar, retry başlar, cache bypass edilir, mesaj queue’da stale kabul edilip düşürülür, rate limit tetiklenir.

Somut pratik karşılıklar:

- Timeout bütçeleme (deadline propagation) – her servise üst seviyeden gelen toplam bütçeden pay ayrılması
- Chaos engineering / fault injection – worst-case senaryonun fiilen üretilmesi
- Latency budget dokümantasyonu – her serviste SLA’nın izole değil, zincirdeki payı olarak tanımlanması

FMEA – Arıza Modu ve Etki Analizi

Otomotivde her parça için sistematik olarak sorulur: bu nasıl bozulur, bozulursa ne olur, bunu nasıl tespit ederiz, önlemi nedir. Bu disiplin yazılımda genelde postmortem olarak reaktif şekilde yapılır; FMEA ise proaktiftir.

FMEA Tablosu Mantığı

Format: Bileşen → Arıza Modu → Etki → Şiddet (S) → Görülme Olasılığı (O) → Tespit Edilebilirlik (D) → RPN (Risk Priority Number = $S \times O \times D$) → Önlem.

Ödeme Akışı Örneği

Ödeme sağlayıcı API çağrısı: Arıza modu – API timeout veriyor ama işlem sağlayıcı tarafında başarılı oldu. Etki – müşteriden para çekildi, sipariş başarısız görünüyor, tekrar denenirse çift çekim. Önlem – idempotency key zorunlu kılmak, timeout durumunda önce sorgula sonra tekrar dene akışı.

Stok düşürme servisi: Arıza modu – ödeme başarılı ama stok düşürme servisi ayakta değil. Etki – iki müşteri aynı son ürünü almış sanıyor. Önlem – ödeme ve stok düşürmeyi tek bir saga/transaction olarak tasarlamak, başarısızsa otomatik refund tetiklemek.

Onay maili gönderme servisi: Arıza modu – mail servisi çöktü. Etki – sipariş başarılı ama müşteri bilmiyor, destek yükü artıyor. Önlem – mail gönderimini kritik yoldan çıkarmak (async kuyruk + retry), sipariş durumunu her zaman DB’den okunabilir kılmak.

Fark Ne Yaratıyor

Postmortem yaklaşımında bu senaryolar ayrı ayrı yangınlar olarak öğrenilir. FMEA yaklaşımında kod yazılmadan önce masa başında çıkarılır; RPN skoru yüksek olanlar tasarıma baştan gömülür, düşük olanlar bilinçli kabul edilebilir risk olarak işaretlenir.

Bathtub Curve – Arıza Oranının Zaman Profili

Arızalar zamana göre kuvvet eğrisi izler: erken dönemde üretim kusurlarından kaynaklanan yüksek arıza oranı (infant mortality), sonra uzun bir düşük-arıza platosu, sonra yaşlanmayla artan arıza oranı (wear-out). Yazılımda bir servisin infant mortality dönemi (yeni deploy sonrası ilk 48 saat) ile wear-out dönemi (memory leak, log dosyası şişmesi, sertifika süresi dolması) farklı izleme stratejileri gerektirir.

Hızlandırılmış Yaşam Testi (Accelerated Life Testing)

Bir aracın 10-15 yıl dayanması beklenir ama bu gerçek zamanda test edilemez; sıcaklık döngüleri, titreşim, nem gibi stres faktörleri yoğunlaştırılarak yılların aşınması haftalara sıkıştırılır. Yazılımda “veritabanı 2 yıl büyürse index performansı nasıl bozulur”, “cache 18 ay çalışırsa memory fragmentation nasıl birikir” gibi uzun-vadeli degradasyon simülasyonu neredeyse hiç yapılmaz – bu, nadir uygulanan ama somut bir disiplindir.

Crash Test Felsefesi

Crash test senaryolarının çoğu normal kullanımda karşılaşılmayan durumlardır; mühendislik bütçesinin büyük kısmı, istatistiksel olarak nadir ama sonucu felaket olan senaryolara ayrılır çünkü risk değerlendirmesi olasılık $\times$ şiddet ile yapılır. Yazılımda nadir-ama-felaket senaryoları (migration sırasında yarıda kesinti, iki region’ın aynı anda failover’a girmesi, yetkilendirme bug’ının tüm kullanıcı verisini açığa çıkarması) genelde “zaten olmaz” diye es geçilir. Bu senaryolar olasılığa

göre değil şiddete göre önceliklendirilip kasıtlı olarak test edilmelidir (chaos engineering, game day tatbikatları).

Üretim / Lean Disiplini

Kaizen ve Andon Cord

Toyota'nın bant hattında herhangi bir işçi bir sorun fark ettiğinde ipi çekip tüm hattı durdurabilir – hiyerarşiden bağımsız olarak hatayı yayılmadan durdurma ilkesi. Yazılım karşılığı, herhangi bir mühendisin seniority'sine bakılmaksızın şüpheli bir deploy'u durdurma yetkisine sahip olmasıdır. Bu yetki çoğu şirkette resmi olarak var ama kültürel olarak yoktur; junior mühendis “belki ben yanlış anlıyorumdur” diye susar, hata büyür.

Poka-Yoke – Hata Geçirmezlik Tasarımı

Otomotivde bir parça yanlış yöne monte edilemeyecek şekilde tasarlanır; hata insan dikkatine bırakılmaz, fiziksel olarak imkansız kılınır. Yazılımda karşılığı, type system'in yanlış kullanımı derleme zamanında engellemesi, API'lerin yanlış kullanılmayacak şekilde tasarlanmasıdır (branded type kullanmak, null yerine Option/Result tipleri). “Dokümantasyonda yazıyor” yaklaşımı poka-yoke değildir; poka-yoke hatayı teknik olarak imkansız kılmaktır.

Just-in-Time ve Envanter Maliyeti

JIT, stoğu minimumda tutarak sermaye verimliliği sağlar ama bedeli kırılganlıktır – bir tedarikçi aksarsa tüm hat durur. Yazılımda minimum bağımlılıkla, her şeyi on-demand çağıran mikroservis tasarımı JIT'e benzer: kaynak verimlidir ama upstream servis yavaşladığında tüm zincir kilitlenir. Otomotivin evrimi öğreticidir: saf JIT'ten “JIT + stratejik buffer stok”a geçilmiştir; yazılımda karşılığı kritik noktalarda bilinçli cache/queue buffer'ı bırakmaktır.

İstatistiksel Süreç Kontrolü (SPC)

Üretim hattında belirli aralıklarla örnekleme yapılır ve ölçümler kontrol grafiklerine işlenir; amaç bir parametrenin kontrol sınırlarına doğru kaymaya başladığını, henüz hatalı ürün çıkmadan önce yakalamaktır – trend hatadan önce gelir. Yazılımda bunun karşılığı, hata oranı sıfır olsa bile p99 latency'nin veya memory kullanımının yavaş yavaş yukarı kayan trendini izlemektir. Çoğu alerting sistemi eşik-bazlıdır; SPC mantığı normal varyasyon aralığının dışına kayma eğilimini eşik geçilmeden önce izler.

Kaizen Blitz

Bir süreç sorununu haftalarca süren toplantılarla değil, 3-5 günlük yoğun çapraz fonksiyonlu bir odaklanmayla çözme pratiğidir. Yazılımda teknik borç temizliği genelde sürekli ertelenen bir backlog maddesi olarak kalır; kaizen blitz mantığı belirli bir acı noktasını bir hafta boyunca tüm ekibin odaklandığı tek konu haline getirip kökten çözmektir.

Six Sigma DMAIC Döngüsü

Kalite sorunları beş adımlı sabit bir döngüden geçirilir:

- Define – sorun tam olarak ne, başarı ölçütü ne
- Measure – mevcut durum sayısal olarak nedir
- Analyze – kök neden nedir

- Improve – çözüm uygulanır
- Control – çözümün kalıcı olduğu sürekli izlenir

Yazılımda bir performans sorunu genelde doğrudan Improve adımına atlanarak çözülür; Measure ve Control atlanır. DMAIC’in somut katkısı, Control adımının ayrı ve zorunlu olmasıdır – bir fix’in kalıcılığını doğrulayan bir metrik olmadan “düzeltildi” denemez.

Kök Neden – 5 Neden (5 Whys)

Bir arızada yüzeysel sebeple durmayıp art arda “neden” sorarak kök nedene inmeyi zorunlu kılan teknik. Yazılım postmortem’lerinde en yaygın hata ilk bulunan teknik nedende durmaktır; beşinci seviyede genelde teknik değil süreçsel bir kök neden bulunur.

Organizasyonel Yapı

Tier 1/2/3 Tedarikçi Modeli

OEM doğrudan binlerce parçayı yönetmez; Tier 1 tedarikçilerle çalışır, onlar da kendi Tier 2/3 zincirlerini yönetir. Her katman kendi kalite sorumluluğunu taşır, OEM sadece arayüz sözleşmesini denetler. Yazılımda karşılığı platform ekibi ile ürün ekipleri arasındaki ilişkidir; platform ekibi her ürün ekibinin iç detayını denetlemez, sadece API sözleşmesini ve SLA’yı garanti eder.

Eşzamanlı (Simultane) Mühendislik

Eski otomotiv modeli sıralıydı; modern otomotivde tasarım, üretim mühendisliği, tedarikçi ve kalite ekibi tasarım henüz donmadan aynı masada oturur. Yazılımda “önce backend biter, sonra frontend entegre eder, sonra QA test eder” hâlâ yaygın bir anti-pattern’dır. Contract-first API tasarımı (OpenAPI spec’inin en baştan ortak referans olarak donması) bunun karşılığıdır.

Kırmızı Takım / Yeşil Takım Ayrımı

Aracı tasarlayan mühendisler ile onu doğrulayan test mühendisleri genelde ayrı raporlama hatlarına bağlıdır – aynı kişi hem tasarlayıp hem yeterince test ettim demek yapısal bir çıkar çatışmasıdır. Yazılımda “kodu yazan kişi kendi testini yazar ve kendi PR’ını merge eder” hâlâ yaygındır; kritik sistemlerde review/onay otoritesinin yazardan bağımsız olması gerekir.

Design for Manufacturing (DFM)

Bir parça sadece işlevsel olarak doğru değil, aynı zamanda seri üretimde tekrarlanabilir olmak zorundadır; mühendislik ile üretim ekibi tasarım aşamasında birlikte çalışır. Yazılımda karşılığı “Design for Operability” – bir mimarının işlevsel doğruluğu değil, deploy edilebilirliği ve gözlemlenebilirliği de tasarım kriteri olmalıdır; operasyonu üstlenecek ekip işin bittiğinde değil en baştan devreye girmelidir.

Standardizasyon ve İzlenebilirlik

OBD-II ve Standartlaştırılmış Hata Kodları

Her arıza üreticiden bağımsız standart bir kod sistemiyle tanımlanır; herhangi bir tamirci herhangi bir markanın aracına bağlandığında aynı dili konuşur. Yazılımda hata yönetimi genelde şirket içinde bile standart değildir; her servis kendi log formatını üretir. Hata taksonomisinin organizasyon çapında standartlaştırılması debugging hızını ve yeni katılan mühendisin adaptasyonunu doğrudan etkiler.

Fonksiyonel Güvenlik Standartları (ISO 26262 / ISO SAE 21434)

ASIL seviyeleri risk büyüklüğüne göre farklı geliştirme titizliği talep eder; fren sistemi ile radyo yazılımı aynı süreçten geçmez. Bağlı araç güvenliği standardı ISO/SAE 21434, tehdit modellemeyi ve saldırı yüzeyi analizini tasarımın ayrılmaz parçası kılar, sonradan eklenen bir katman değil. Yazılımda risk bazlı süreç farklılaştırması nadirdir; güvenlik çoğunlukla geliştirme bitince pentest yaptırılarak sona bırakılır. Threat modeling sprint planlamasının bir parçası olmalı, ayrı bir güvenlik sprinti değil.

Parça Numaralandırma ve Sürüm İzlenebilirliği

Her parçanın, her revizyonun kendine özgü bir numarası vardır; bir aracın hangi VIN’inde hangi parça revizyonunun kullanıldığı tam olarak izlenebilir. Yazılımda “hangi kullanıcılar şu anda hangi feature flag kombinasyonu, hangi API versiyonuyla çalışıyor” sorusu çoğu sistemde kolay cevaplanamaz. Request-level provenance loglaması, bir incident sonrası “bu hangi deploy’da başladı” sorusunu dakikalar içinde cevaplamayı sağlar.

Homologasyon

Bir aracın her pazara girebilmesi için o pazarın regülasyonlarına göre ayrı test/sertifikasyon sürecinden geçmesi gerekir. Yazılımda bir özelliğin farklı bölgelerde (GDPR, KVKK, CCPA gibi farklı veri regülasyonları) farklı doğrulama gerektirmesi aynı mantığa denk gelir; “tek kod her yerde aynı davranış” varsayımı naiftir.

Kaynak Yönetimi

Ağırlık Bütçesi Yönetimi

Bir aracın toplam ağırlık hedefi vardır; her parça takımı kendi ağırlık payını alır ve bunu aşarsa gerekçelendirmek zorundadır. Yazılımda karşılığı performans bütçesi olabilir – sayfa yükleme süresi, bundle boyutu, API latency bütçesinin takımlar arasında açıkça dağıtılması. Çoğu organizasyonda bu bütçe ya hiç yok ya da tek bir takımın sorumluluğuna bırakılmıştır.

Tedarik Zinciri Redundansı

Çip krizinde tek bir tedarikçiye bağımlı olmanın bedeli ağır ödendi; kritik parçalarda çoklu tedarikçi standart hale geldi. Yazılımda karşılığı, tek bir üçüncü parti servise (ödeme sağlayıcı, SMS API’si, cloud provider) sert bağımlılık kurmanın riskidir; kritik bağımlılıklarda adapter pattern ile ikinci sağlayıcıya geçiş süresini önceden test edilmiş halde tutmak gerekir.

Değer Mühendisliği

Her parça periyodik olarak “bu fonksiyon gerçekten bu maliyeti hak ediyor mu” sorgusundan geçer; gereksiz güç, gereksiz tolerans, gereksiz malzeme kalitesi ayıklanır. Yazılımda artık kullanılmayan abstraction katmanları, “belki lazım olur” diye eklenen esneklik birikir ve bu genelde sadece teknik borç adı altında kriz anında sorgulanır.

Geri Besleme Döngüleri

Garanti Verisi – Üretimden Tasarıma Geri Besleme

Satış sonrası garanti kapsamındaki arızalar sistematik olarak toplanır ve bir sonraki model yılının tasarım revizyonuna doğrudan girdi olur. Yazılımda müşteri destek talepleri genelde ayrı bir

sistemde yaşar ve mühendislik ekibine sistematik geri beslenmez; sahada en çok tekrar eden sorunların düzenli ve veri odaklı şekilde roadmap'e bağlanması gerekir.

Dijital İkiz

Modern otomotiv geliştirme, fiziksel prototip üretmeden önce aracın tamamını dijital ortamda simüle eder; fiziksel prototip, dijital modelin zaten yüksek güvenle doğru olduğu noktada üretilir. Yazılımda staging/shadow ortamları bunun karşılığıdır, ama çoğu ekipte staging production'ın küçültülmüş ve gerçekçi olmayan bir kopyasıdır. "Staging'de çalıştı" ile "production'da çalışır" arasındaki fark genelde bu eşdeğerliğin eksikliğinden kaynaklanır.

Kill Switch ve Recall Kültürü

Bir güvenlik kusuru bulunduğunda recall mekanizması itibar kaybı olsa da hızlı ve şeffaf işletilir çünkü ertelemenin bedeli çok daha ağırdır. Yazılımda feature flag'ler teknik olarak recall imkanı sağlar, ama kültürel olarak çoğu ekip bir özelliği geri çekmeyi başarısızlık kabulü gibi görüp geciktirir.

Tork Spesifikasyonu ve Konfigürasyon Sürüklenmesi

Her cıvatanın belirli bir tork değeriyle sıkılması gerekir; bu değerler dokümanite edilir ve montaj hattında otomatik doğrulanır, insan takdirine bırakılmaz. Yazılımda karşılığı config drift'tir – environment variable'ların, feature flag durumlarının, resource limit'lerin "birileri elle ayarladı, sonra unutuldu" şeklinde sürüklenmesi. Kritik parametrelerin insan hafızasına değil infrastructure-as-code ve config validation pipeline'ına bağlanması gerekir.

Kozmetik/Algısal Kalite (NVH)

Bir aracın motoru teknik olarak mükemmel çalışabilir ama kapı kapanış sesi ucuz hissettiriyorsa algılanan kalite düşer; NVH mühendisliği bunu ayrı bir disiplin olarak ele alır. Yazılımda karşılığı, sistemin teknik olarak doğru ama kullanıcı deneyimi olarak ucuz hissettiren tarafıdır – buton tıklama gecikmesi, loading spinner'ın akıcılığı, hata mesajının tonu.

Ortak Alttema

Otomotiv mühendisliğinin altı ekseninde ortaya koyduğu tüm disiplinler tek bir ilkenin farklı tezahürleridir: hatanın maliyeti geç fark edildikçe katlanarak artar, bu yüzden tasarım aşamasına orantısız yatırım yapılır. Yazılımın "iterasyon ucuzdur, sonra düzeltiriz" kültürü çoğu durumda doğru bir varsayımdır, ama kritik sistemlerde (ödeme, kimlik doğrulama, veri bütünlüğü, güvenlik) bu varsayım yanlıştır. Otomotivin öğrettiği asıl ayırım, her şeye aynı titizliği uygulamak değil, hangi bileşenin bu titizliği hak ettiğini baştan belirlemektir.